

Assignment 02

Due: 2026-07-08, 01:00pm

2 Locked in Tasks Freedoms no more

Reminder: **You have to pass every assignment in order to qualify for the exam.** This one once again revolves around practical applications of multicore programming using C++. However, for your analysis it is highly recommended to resort to Python (Pandas, Numpy, Matplotlib). Also, Bash scripts might be useful for conducting measurement campaigns.

This assignment is accompanied by `additional-material.pdf`.

2.1 General information

You can still use “your” CIP (given by your group's name) to carry out the experiments. Regarding the group name, behind the slash, you can find the room number where they are located in building 93. Alternatively, you may log into them remotely via SSH:

```
ssh USERNAME@cipNUMBER.informatik.uni-osnabrueck.de
```

Your Python code does not need CIP compatibility, so you might also just run it on your local machine, if you wish. However, please provide information about your Python environment then (Python version, package list).

We will be using vanilla C++20 (with `#pragma once`) and you *must* only use the STL (standard library, so no *Boost* etc.). Check `Cppreference`¹ for a general documentation and the scope of C++20.

- Your program *must* compile successfully with (no more, no less):
`-std=c++20 -O2 -Wall -Wextra -pedantic -Wno-interference-size`
on GCC 12.3 or newer (**CIPs need to invoke `g++-12` instead of `g++`**).
- Compiling with these options *must not* produce any warnings.
- You *must not* add additional C++ source files other than the ones given.
- You *should* use a consistent programming style with consistent indentation.
- You *should* comment your code meaningfully and explain your design decisions. I.e., do not comment for the sake of commenting (e.g., no `// print to stdout` etc.), but do not leave your code uncommented either.

Failing the “must”-requirements will result in a failed assignment. Failing the “should”-requirements will result in a deduction of points (having less than 50 % of the points leads to failure). Of course, wrong software behavior will also lead to a deduction of points.

You *should* also avoid C-style features and raw C++ operations:

¹<https://en.cppreference.com/>

Dynamic Arrays Use `std::vector<T> data(n)` instead of `T* data = new T[n]`. If you encounter errors with `std::vector`, try using `std::deque`, which does not move or copy around the objects it stores (a vector must do so on resizing).

Printing Use `std::cout` for standard printing and `std::cerr` to output error messages. Do not use `printf` etc., as these functions are from C and mostly not type-safe. If you need thread-safe printing, you may use `std::ostream(std::cout) << ...`² located in the `<ostream>` header.

Type-Casting Do not cast in C-style, `T1 a; T2 b = (T2)a;`. Instead, use `static_cast<T2>(a)`. If you use polymorphism and want to cast objects to classes down the inheritance hierarchy, then use `dynamic_cast<T2>(a)`.

Threading Use `std::jthread`, not `std::thread`. It primarily enforces joining the thread upon destruction of the corresponding object³.

References There is a saying: “Use references wherever you can, and pointers whenever you must.” A reference must point to an object, so if your function must read a complex value (e.g. a class instance), it should be passed as constant reference: `void foo(const object_t& obj)`. Omit the `const`, if you need write-access.

Pointers As mentioned earlier, only use them, if you really need to. Pointers might point to nothing (i.e. `nullptr`), so you must check for that.

Error Handling Handle your edge-cases and errors accordingly, i.e. do not forget about error handling. You can choose, whether you use exceptions or returned values to indicate failure; even case-by-case.

Memory Management All allocated memory (new and alike) must be freed (`delete` and alike) when it is no longer needed. This is usually the case, when all pointers/references to this memory region go out of scope. In order to avoid forgetting about this crucial responsibility, C++ provides so-called *smart pointers*. `std::unique_ptr<T>` is the sole owner of the memory it points to. It cannot be copied, only referenced and moved (the latter can be thought of as *ownership transfer*). So whenever this pointer goes out of scope, it frees the memory along with it. `std::shared_ptr<T>` uses reference counting to keep track of how many other shared pointers refer to this memory region. As soon as all shared pointers to it are destroyed, the memory will be freed automatically. Furthermore, C++20 offers `std::atomic<std::shared_ptr<T>>` implementations, thus dynamically allocated memory can be safely and linearizably shared across threads.

C++ has a myriad of pitfalls and these rules are here to enforce some degree of stability.

²https://en.cppreference.com/cpp/io/basic_ostream

³<https://stackoverflow.com/q/62325679/5953061>

2.2 Assignment steps

These steps are grouped by the lecture introducing the mentioned locks. The first lecture covers the implementations TAS to *Anderson Queue*, whereas the second one adds MCS (and more; but we do not investigate these). Furthermore, it includes a lock implementation from exercise 09, called “ALog”.

2.2.1 First lecture on slide set 07

A first research question Slide set 07 explains a variety of Lock implementations, and – once again – makes claims regarding their efficiency. Of course, this cannot remain untested, so you phrase a *crisp and precise* research question that questions the performance order in Figure 2 (it is an amalgamation of the conceptual plots from the lecture, such that it orders all of them with each other). So, you are primarily interested in whether their runtime order – e.g., TTAS is strictly more performant than TAS etc – holds true. We do not investigate the runtime growth, such as “linear”, “quadratic” etc.

You could also create an overarching, more abstract research question first, followed by more concrete sub-research questions building on it.

Basic setup All of these locks implement the Lock interface, which makes it ideal for re-using the same testing environment for all locks. So, the virtual nature of the `lock()` and `unlock()` member functions enables changing their behavior during runtime through polymorphism. Your `main()` must parse key-value command-line parameters, i.e. pairs like `--key value`. Furthermore, the order in which these pairs are specified should not matter, e.g., both

- `--key0 value0 --key1 value1`
- `--key1 value1 --key0 value0`

must specify the same input. As of now, your program must accept the following arguments (`<std::size_t:1..n>` means “a `std::size_t` larger than 0”, the angular brackets are never part of the input. You may assume `std::size_t` is unsigned long long.):

--jobs `<std::size_t:1..n>` The amount of concurrent threads trying to access the same lock.
Default: 1.

--samples `<std::size_t:1..n>` In order to aggregate your data, your test needs repetitions. This argument specifies the amount of runtime samples – i.e. experiment repetitions – the program has to carry out. **Default:** 1.

--lock `<std::string:LOCK_ID_STRING>` The internal name of the lock to use for the experiments. Their names will be *prescribed* throughout this assignment. **No default value.**

Naturally, unknown or missing arguments, and invalid values must be treated as an error.

Getting simple locks Now, you begin copying over some of the simple lock implementations:

- Test-and-Set Lock
(TASlock) [`--lock tas`]
- Test-and-Test-and-Set Lock
(TTASlock) [`--lock ttas`]

Use the pre-defined headers, and provide inline implementation, i.e. implement a lock only in its header. E.g., `TASlock.hpp` contains only the implementation of the class `TASlock`. Also, do not add more source files than already provided.

Creating contention A simple way of causing contention is by implementing a ticker that grants atomicity via spinlock-based mutual exclusion instead of using hardware-assisted single-instruction locking (e.g., LOCK instruction prefix on x86). Complete the LockedTicker class in Ticker.hpp, such that a `ticker++` operation – `.fetch_add(1)` basically – is protected by the specified lock. It should accept an externally created Lock (Lock.hpp) reference that is used internally for mutual exclusion. This Lock reference will naturally be able to bind to any of Lock's child-classes' instances.

In order to have some kind of general baseline, you have to complete STLatomicTicker⁴ in Ticker.hpp as well. It refrains from using locks explicitly and just wraps `std::atomic_size_t`. If

`--lock atomic`

is given, STLatomicTicker must be used instead of LockedTicker.

Running experiments Your program must perform the following steps:

1. Parse the command-line arguments
2. Instantiate the correct child-class of Lock, according to `--lock`.
3. Repeat these steps `--samples-many` times:
 - a) Create the correct Ticker object that binds to the previously created Lock sub-class instance.
 - b) Measure the elapsed time (in nanoseconds) for:
 - i. Spawn `--jobs-many` threads each of which shares this Ticker object, and execute `ticker++` 10^5 times in a loop. If you store your threads inside a container, make sure, no container allocation is part of the measured time.
 - ii. Wait until all threads have finished (joining).
 - c) Print `<--jobs>, <--lock>, <measured runtime in nanoseconds>` as a new line to `stdout`⁵. Again, `<>` are not part of the output, just delimiters to express the pattern.

Hence, your program must output each sample as a single line of a CSV "file" (just printed to the terminal). So, e.g.,

`./locks --lock tas --jobs 8 --samples 4`

will produce a terminal output looking like `example-run-result.csv`. This is handy, as the output can be redirected to any file via piping in, e.g., Bash:

```
1 # create/truncate CSV with header with > operator
2 echo "jobs,lock,runtime_ns" > my_csv_file.csv
3 # append to CSV with >> operator
4 ./locks --lock tas --jobs 8 --samples 4 >> my_csv_file.csv
```

Running a campaign Now that you can repeat individual experiment setups, you must compose it into a whole campaign allowing you to obtain the data of interest. Write a script (preferably in Bash) that runs all combinations of the parameter sets expressed in Table 1.

⁴Since atomicity must hold for all Ticker implementations and `std::atomic` does not guarantee lock-freedom, it was assigned this name.

⁵`std::cout` uses `stdout`, whereas `std::cerr` uses `stderr`.

lock	jobs	samples
{ atomic, tas, ttas }	× { 1, 2, 4, 6, 8, 10 }	× { 50 }

Table 1: Initial parameter combinations

Make it create a single CSV per lock (`--lock>.csv`).

Then, run this campaign on a machine with at least 10 hardware threads⁶. Like last time, you have to specify your machine with the CPU count, the memory amount, the compiler version, and the operating system. It is recommended to use the CIPs, and then also provide its ID.

Plotting a campaign Use the output CSVs to create a plot which illustrates the average end-to-end runtime (i.e. the mean of the `runtime_ns` column) per job count with error bars (similar to Figure 1). Provide a script that does this for a list of filepaths to output CSVs from the previous step, so one file corresponds to one plot in the coordinate system.

What is the difference in perspective/setup between this plot and the one[s] from the slides (see Figure 2)?

What could one do to obtain a similar perspective without having to re-run the campaign?

Adding support for Backoff Locks Another improvement for a test-and-set lock was introduced by adding a backoff mechanism making the lock refrain from actively acquiring the lock exponentially longer. It, however, comes with two parameters (`MIN_DELAY` and `MAX_DELAY`) needing to be tuned. Therefore, add these two additional command-line arguments:

--backoff-min `<int:0..n>` `MIN_DELAY` in nanoseconds. **Default: 0 or optimum.**

--backoff-max `<int:0..n>` `MAX_DELAY` in nanoseconds. **Default: 0 or optimum.**

Make sure, the minimum is never larger than the maximum.

When `--lock backoff` is provided, these parameters have to influence the lock's behavior accordingly. If some other lock is specified, you may either ignore these arguments or raise an error.

For the random-number generator, either use `std::mt19937` or `std::mt19937_64` with a uniform distribution (how-to example is here⁷). If you encounter strange results with thread-sleeping that blocks, try to “actively” sleep instead.

As for the output, make the sample results with `--lock backoff` look like this:

```
<--jobs>,backoff-<--backoff-min>-<--backoff-max>,<MEASURED_RUNTIME>
```

So, the lock name also includes these parameters in the CSV; this makes plotting with the same script easier. `example-backoff-tuning.csv` provides an example for `--backoff-min 100`, `--backoff-max 800`.

Create an additional campaign script that explores several combinations of min-max backoff pairs with 50 samples each. Visualize the five most promising candidates in the same *kind* of plot from earlier.

Why did you choose to explore this particular set of combinations?

Pick the best-performing min-max pair, and add it to the plot juxtaposing all previous locks. Also, change the default values of `backoff-min/-max` accordingly.

Explain why this particular combination won.

⁶`lscpu` calls them “CPU(s)”. This is the product of “Socket(s)”, “Core(s) per Socket”, and “Thread(s) per core”.

⁷https://en.cppreference.com/cpp/numeric/random/uniform_int_distribution

lock a	!=	<	>	lock b	jobs
atomic	False	True	False	tas	1
...					

Table 2: Shape of the hypothesis-testing result table. The columns !=, <, > correspond to the *alternative hypothesis*. So, != is true, if the null hypothesis (=) was rejected between locks a & b, and so on. Sort the table in this order: (1) lock a, (2) lock b, (3) jobs.

Anderson Queue Lock Add the ALock (--lock aq) as part of ALock.hpp. However, the implementation from the slides contains a flaw which its slides explain themselves: it is not cacheline-aligned. Also, it currently requires a compile-time amount of threads to be specified.

Modify the code from the slides, such that the supported amount of threads is specified during runtime. So,

```
lock = ALock<N>();
      becomes
lock = ALock(N);
```

Modify the code further, such that *each* element in the flags attribute is cacheline-aligned for the system, you run the experiments on. Thereby, each element should reside in its own cacheline.

Obtain this lock's result-CSV in a similar fashion to the previous locks, and add its plot to the overview figure. To complete your artifacts, you also have to add this lock to the campaign script.

Evaluating a campaign Now, write a script that tests – using hypothesis testing (see [additional-material.pdf](#)) – whether the performance order in Figure 2 holds true (95 % confidence). Make it read the CSVs from the measurement campaign. Assume that --lock atomic is going to be the ideal case.

Which hypothesis test did you choose, and why?

The script should then test all pairs (but not a lock with itself) of implemented locks for each job count. It should produce a CSV with the layout of Table 2, but again written to stdout.

Answer the research question from the beginning for the current locks. Explain your decision using these results. You could find a way to visualize / digest these results, so your explanation becomes clearer.

2.2.2 Second lecture on slide set 07

Including new locks Add the ALog (--lock alog) (from exercise sheet 09⁸) and MCSlock (--lock mcs) to your code base, and obtain their respective result-CSVs in a similar fashion to the previous locks. To complete your artifacts, you also have to add these locks to the campaign script.

Update the overview figure by adding the two new plots to it. Re-run your hypothesis-testing script on all locks, and include the added locks in your research-question answer.

⁸That is the name used there. If you missed the solution, you can find it – i.e. you search – under its official name, which is one of the locks mentioned here: <https://en.wikipedia.org/w/index.php?title=Spinlock&oldid=1315545697>. As 64 bit can encode quite enough numbers, a memory complexity of $\mathcal{O}(1)$ is typically assumed instead of logarithmic complexity.

What is your proposed performance order for all locks? For example,

$$\text{atomic} \leq \text{aq}, \text{mcs}, \text{ttas} \leq \text{tas}$$

would mean that `atomic` is better or equal than `aq`, `mcs`, `ttas`, and `tas`. At the same time, no lock is worse than `tas`, while `aq`, `mcs`, `ttas` have no statistically significant difference among each other. You might need to answer this by thread count, if necessary, or provide multiple such relation lines.

Questioning your own work After presenting your results, you should also discuss the limitations of your approach.

Do you agree with the logic of the core program for answering the research question? Is it reproducible, given an identical machine?

Do you think, your machine should have disabled/enabled some system settings (BIOS/UEFI/OS)? If so, which would that be?

Are there new or still some open questions?

What else is there to note?

2.3 Submitting your results

As last time, **fill in the Vips forms on StudIP**, where you again also have to **upload your artifacts**:

- Your entire source code
 - core program (`main.cpp` and the headers)
 - plotting script
 - campaign script
 - backoff-exploration campaign script
 - evaluation script

No pre-compiled binaries and no unnecessary meta files (e.g., .DS Store from macOS).

- The campaign results
 - the CSVs with all samples resulting from the campaign with Table 1, *and* your chosen backoff implementation, also with 50 samples, so
 - * `atomic.csv`
 - * `tas.csv`
 - * `ttas.csv`
 - * `backoff.csv`
 - * `aq.csv`
 - * `alog.csv`
 - * `mcs.csv`
 - the overview plot belonging to these CSVs
- The backoff-tuning results
 - in a subfolder, the CSVs from your exploration, 50 samples each

- designate the five most promising results consistently
- the plot belonging to these five CSVs
- The evaluation CSV that corresponds to Table 2.
- Supporting plots that helped you to obtain and explain your findings.

Upload all of this as a single TAR or ZIP file.

Again, you will either get all 100 points, indicating a passed assignment, or less than 50 points (with explanations), indicating a failed assignment. In the former case, you might still receive some comments on your submission. The subtask weights can be found on StudIP as well.

Appendix

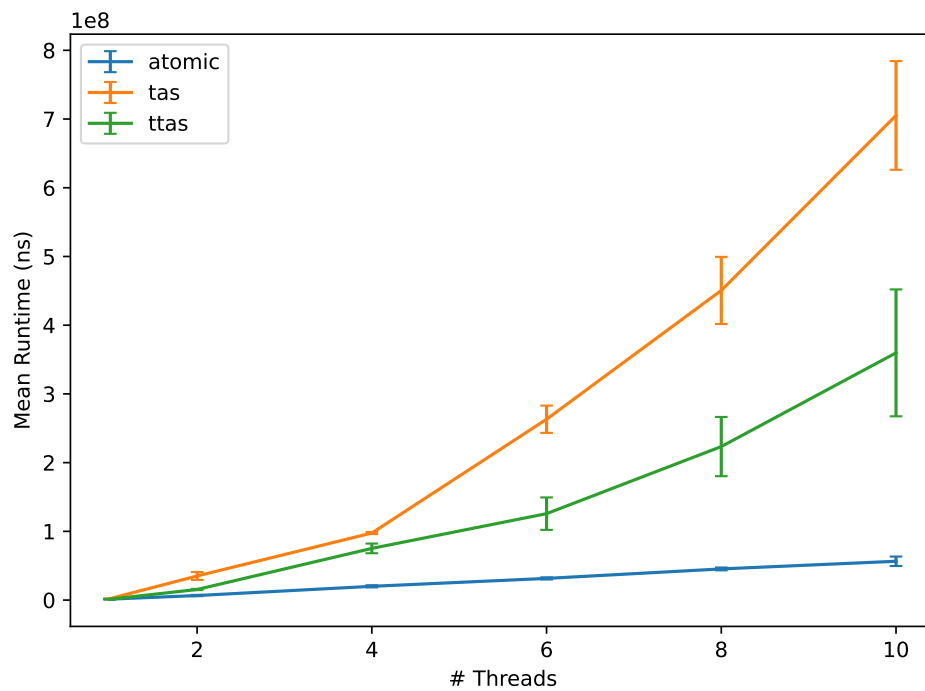


Figure 1: Example Plot: Mean total runtime per amount of threads operation on the same atomic Ticker ($N = 50$).

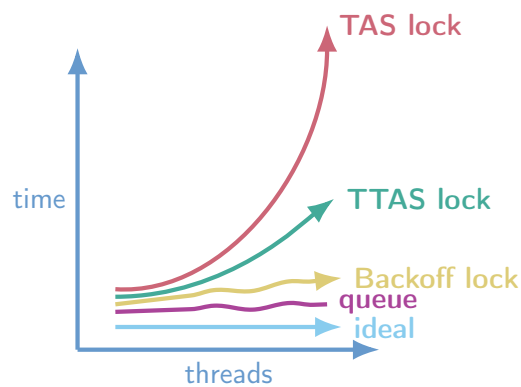


Figure 2: Expected trends (for a fixed workload), according to the slides. “queue” represents the queuing locks.